



Probabilistic Machine Learning

16. Exemplar-based Methods

김무진 / '26.04.02

17. Kernel Method(Support Vector Machine)



Computational Data Science LAB



CONTENTS

1. Exemplar-based-Methods
 2. Kernel Methods
 3. Support Vector Machine
- 
- 

01 | Exemplar-based-Methods

KNN

- Exemplar-based Methods
 - ✓ 지금까지는 모수적 모델에 집중
 - ✓ 훈련 데이터를 유지하는 비모수적 모델을 고려
 - ✓ 모델은 테스트 입력 x 와 각 훈련 예제 x_n 사이의 유사도 측면에서 정의할 수 있는 것에 집중

- KNN

- ✓ 가장 가까운 K개의 데이터의 라벨로 결정
- ✓
$$p(y = c | \mathbf{x}, \mathcal{D}) = \frac{1}{K} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n = c)$$

- ✓ 간단하며 분포 가정이 필요하지 않음
- ✓ 통계적 모델을 정의하지 않고 변수들 사이 복잡한 상호작용을 수집

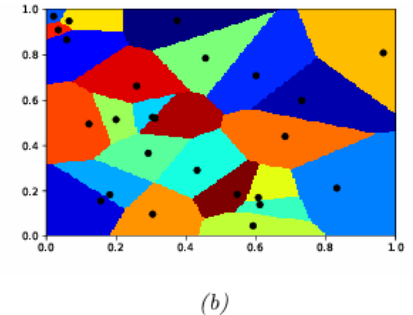
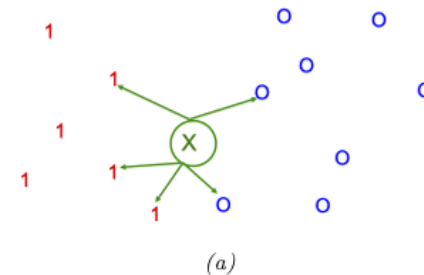
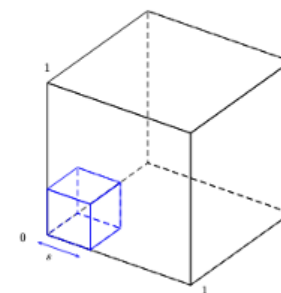


Figure 16.1: (a) Illustration of a K -nearest neighbors classifier in 2d for $K = 5$. The nearest neighbors of test point $\mathbf{x}$ have labels $\{1, 1, 1, 0, 0\}$, so we predict $p(y = 1 | \mathbf{x}, \mathcal{D}) = 3/5$. (b) Illustration of the Voronoi tessellation induced by 1-NN. Adapted from Figure 4.13 of [DHS01]. Generated by [knn_voronoi_plot.ipynb](#).

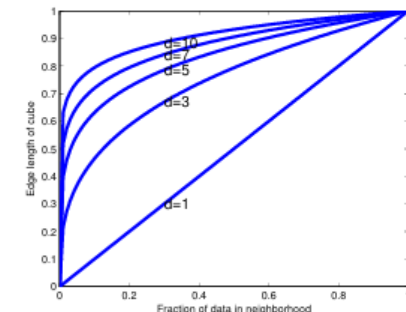
01 | Exemplar-based-Methods

The curse of dimensionality

- The curse of dimensionality(차원의 저주)
 - ✓ 차원이 커질수록 “가까운 이웃” 이 실제로는 전혀 가깝지 않게 됨
 - ✓ 가정
 - 입력 데이터 x 는 D 차원 unit cube 안에 존재
 - 데이터는 균일분포
 - e 는 데이터의 비율 p 를 포함하려면 필요한 큐브의 한 변의 길이
 - ✓ 데이터 비율 = 공간비율 이므로 $e^D = p$
 - ✓ 데이터 p 비율을 포함하려면 cube 한 변의 길이 = $e_D(p) = p^{\frac{1}{D}}$
 - ✓ 10%라 가정, $D = 1 \rightarrow e = 0.1$
 - ✓ $D = 2 \rightarrow e \approx 0.316$
 - ✓ $D = 10 \rightarrow e \approx 0.8$



(a)



(b)

Figure 16.3: Illustration of the curse of dimensionality. (a) We embed a small cube of side s inside a larger unit cube. (b) We plot the edge length of a cube needed to cover a given volume of the unit cube as a function of the number of dimensions. Adapted from Figure 2.6 from [HTF09]. Generated by [curse_dimensionality_plot.ipynb](#).

01 | Exemplar-based-Methods

Kernel Density Estimation(KDE)

- Parametric vs Non-parametric Density Estimation

- ✓ Parametric 밀도추정은 미리 pdf에 대한 모델을 정해놓고 데이터들로 부터 모델의 파라미터를 추정하는 방식
- ✓ Non Parametric 밀도추정은 순수하게 관측된 데이터만으로 확률밀도함수를 추정

- Kernel Density Estimation

- ✓ Non-Parametric 밀도추정 방법 중 하나로, Kernel Function을 이용하여 추정
- ✓ 각 데이터 포인트별로 값 매김할 수 있는 확률분포 $p(x)$ 를 정의
- ✓ Kernel은 데이터 주변의 영향 범위를 결정하는 함수
- ✓ Kernel모양은 각 데이터가 주변에 미치는 영향의 형태를 결정하고, 추정 분포의 매끄러움과 지역성이 달라짐

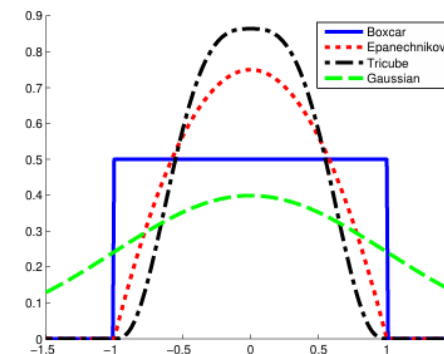


Figure 16.8: A comparison of some popular normalized kernels. Generated by [smoothingKernelPlot.ipynb](#).

- Density Kernels

- ✓ $K : \mathbb{R} \rightarrow \mathbb{R}_+$, $\int K(x)dx = 1$, $K(-x) = K(x) \rightarrow \int xK(x)dx = 0$
- ✓ $\int xK(x - x_n)dx = x_n$

Name	Definition	Compact	Smooth	Boundaries
Gaussian	$\mathcal{K}(x) = (2\pi)^{-\frac{1}{2}} e^{-x^2/2}$	0	1	1
Boxcar	$\mathcal{K}(x) = \frac{1}{2} \mathbb{I}(x \leq 1)$	1	0	0
Epanechnikov kernel	$\mathcal{K}(x) = \frac{3}{4} (1 - x^2) \mathbb{I}(x \leq 1)$	1	1	0
Tri-cube kernel	$\mathcal{K}(x) = \frac{70}{81} (1 - x ^3)^3 \mathbb{I}(x \leq 1)$	1	1	1

Table 16.1: List of some popular normalized kernels in 1d. Compact=1 means the function is non-zero for a finite range of inputs. Smooth=1 means the function is differentiable over the range of its support. Boundaries=1 means the function is also differentiable at the boundaries of its support.

01 | Exemplar-based-Methods

Kernel Density Estimation(KDE)

- Density Kernels

- ✓ 커널의 너비는 대역폭 모수 h 를 도입하여 통제 가능

- ✓ $K_h(x) \triangleq \frac{1}{h} K\left(\frac{x}{h}\right)$

- KDE TO KNN

- ✓ KNN은 KDE의 한 형태로도 유도될 수 있음

- ✓ 클래스 조건부 밀도 $p(x|y=c) = \frac{N_c(x)}{N_c V(x)}$

- ✓ 클래스 사전분포를 $p(y=c) = \frac{N_c}{N}$ 이라 하면, 클래스 사후 분포는 다음과 같이 주어짐

- ✓
$$p(y=c|\mathbf{x}, \mathcal{D}) = \frac{\frac{N_c(\mathbf{x})}{N_c V(\mathbf{x})} \frac{N_c}{N}}{\sum_{c'} \frac{N_{c'}(\mathbf{x})}{N_{c'} V(\mathbf{x})} \frac{N_{c'}}{N}} = \frac{N_c(\mathbf{x})}{\sum_{c'} N_{c'}(\mathbf{x})} = \frac{N_c(\mathbf{x})}{K} = \frac{1}{K} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n = c)$$

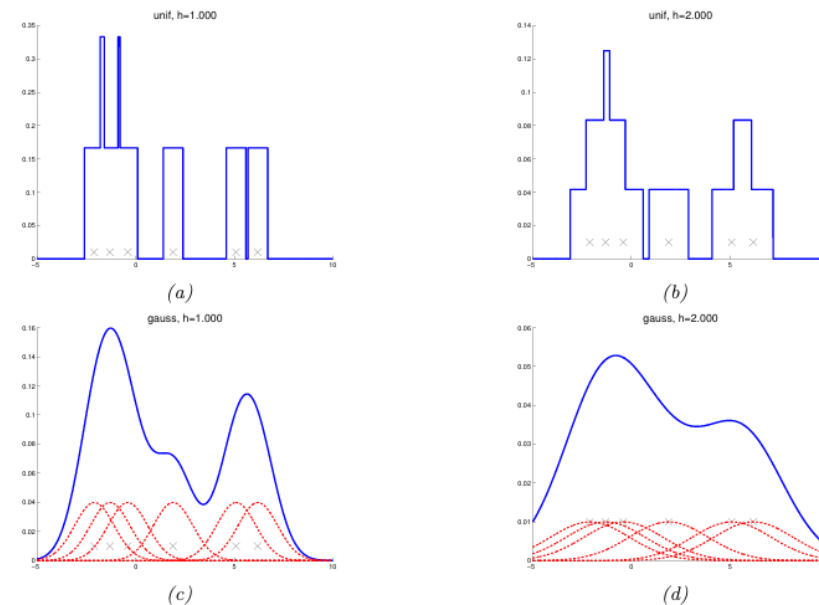


Figure 16.9: A nonparametric (Parzen) density estimator in 1d estimated from 6 data points, denoted by x . Top row: uniform kernel. Bottom row: Gaussian kernel. Left column: bandwidth parameter $h = 1$. Right column: bandwidth parameter $h = 2$. Adapted from http://en.wikipedia.org/wiki/Kernel_density_estimation. Generated by `parzen_window_demo2.ipynb`.

02 | Kernel Methods

Mercer Kernels

- Mercer Kernels

- ✓ 커널 함수는 두 벡터가 얼마나 유사한 지 측정하는 함수
- ✓ N 개의 고유한 점 $x_i \in X$ 의 임의의 집합, 그리고 임의로 선택한 숫자 $c_i \in \mathbb{R}$ 에 대해 다음의 조건을 따르는 임의의 대칭함수
- ✓ $\sum_{i=1}^N \sum_{j=1}^N K(x_i, x_j) c_i c_j \geq 0$

- Gram Matrix

- ✓ $\mathbf{K} = \begin{pmatrix} K(x_1, x_1) & \cdots & K(x_1, x_N) \\ \vdots & & \vdots \\ K(x_N, x_1) & \cdots & K(x_N, x_N) \end{pmatrix}$
- ✓ Gram Matrix가 임의의 집합에 대해 오직 Positive definite matrix(양의 정 부호 행렬)이라면, Mercer Kernel이라 말함
- ✓ $c^T K c \geq 0$

02 | Kernel Methods

Mercer's Theorem

- Mercer's theorem
 - ✓ 어떤 kernel $K(x, x')$ 가 Mercer Kernel이면 그 값은 어떤 feature vector $\phi(x)^T \phi(x')$ 의 내적처럼 쓸 수 있음
 - ✓ Positive definite matrix(양의 정 부호 행렬) K 에 대해 $K = U^T \Lambda U$
 - ✓ $k_{ij} = \left(\Lambda^{\left(\frac{1}{2}\right)} U_{:,i} \right)^T \left(\Lambda^{\left(\frac{1}{2}\right)} U_{:,j} \right)$
 - ✓ $k_{ij} = \phi(x_i)^T \phi(x_j) = \sum_m \phi_m(x_i) \phi_m(x_j)$
 - ✓ 즉, 커널 값은 커널 행렬의 고유벡터를 통해 정의되는 어떠한 특성 벡터의 내적을 수행하여 계산됨
- Why Inner product interpretation?
 - ✓ 내적은 기하적 의미가 존재
 - ✓ 선형 방법을 비선형 문제에 쓸 수 있음
 - ✓ **Kernel Trick**이 가능해짐

03 | Support Vector Machine

SVM Introduction(Hard Margine)

- SVM Introduction

$$\checkmark \min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad \tilde{y}_n(\mathbf{w}^T \mathbf{x}_n + w_0) \geq 1, n = 1 : N$$

Original Problem

$$\text{minimize } \frac{1}{2} \|\mathbf{w}\|_2^2$$

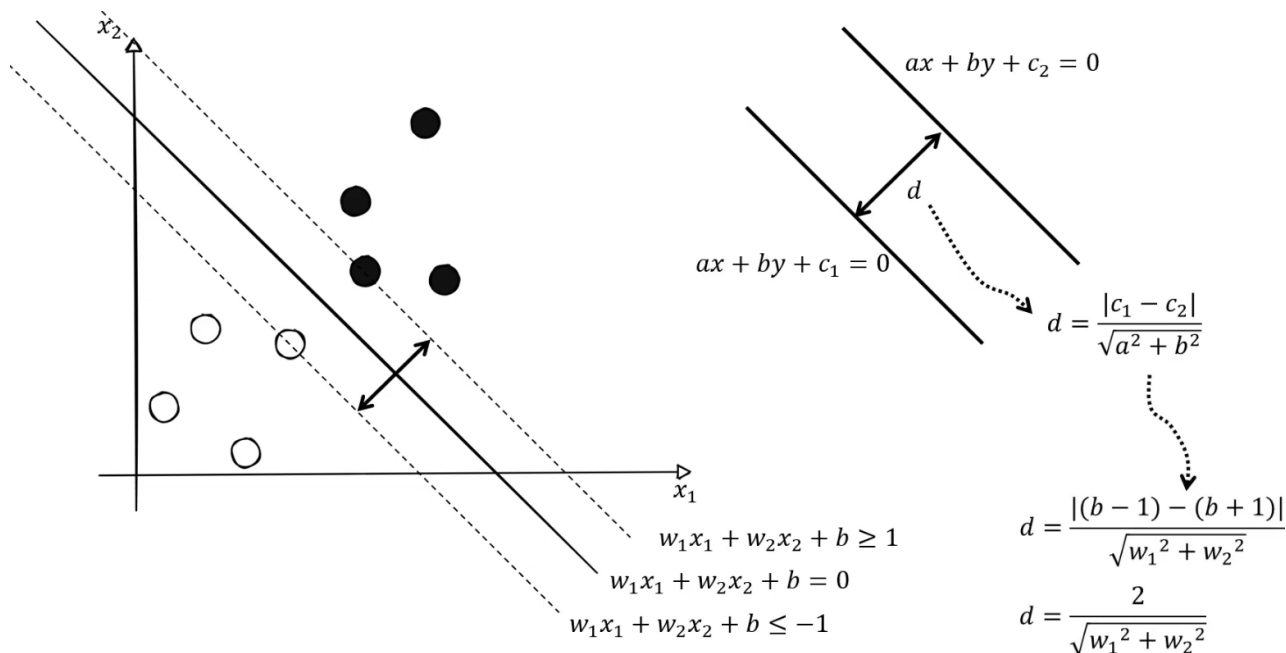
$$\text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, i = 1, 2, \dots, n$$

Lagrangian multiplier를 이용하여 Lagrangian primal문제로 변환

Lagrangian Primal

$$\max_{\alpha} \min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|_2^2 - \sum_{i=1}^n \alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1)$$

$$\text{subject to } \alpha_i \geq 0, i = 1, 2, \dots, n$$



03 | Support Vector Machine

Primal to Dual

• Primal Problem

Lagrangian Primal

$$\max_{\alpha} \min_{w, b} \mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)$$

subject to $\alpha_i \geq 0, i = 1, 2, \dots, n$

Convex, continuous이기 때문에 미분 = 0에서 최소값을 가짐

$$\textcircled{1} \quad \frac{\partial \mathcal{L}(w, b, \alpha)}{\partial w} = 0 \quad \Longrightarrow \quad w = \sum_{i=1}^n \alpha_i y_i x_i$$

$$\textcircled{2} \quad \frac{\partial \mathcal{L}(w, b, \alpha)}{\partial b} = 0 \quad \Longrightarrow \quad \sum_{i=1}^n \alpha_i y_i = 0$$

$$\underbrace{\frac{1}{2} \|w\|_2^2}_{\textcircled{1}} - \underbrace{\sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)}_{\textcircled{2}}$$

$$\textcircled{1} \quad \frac{1}{2} \|w\|_2^2 = \frac{1}{2} w^T w$$

$$= \frac{1}{2} w^T \sum_{j=1}^n \alpha_j y_j x_j$$

$$= \frac{1}{2} \sum_{j=1}^n \alpha_j y_j (w^T x_j)$$

$$= \frac{1}{2} \sum_{j=1}^n \alpha_j y_j \left(\sum_{i=1}^n \alpha_i y_i x_i^T x_j \right)$$

$$= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

RECALL

$$\frac{\partial \mathcal{L}(w, b, \alpha)}{\partial w} = 0 \quad \Longrightarrow \quad w = \sum_{i=1}^n \alpha_i y_i x_i$$

$$\underbrace{\frac{1}{2} \|w\|_2^2}_{\textcircled{1}} - \underbrace{\sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)}_{\textcircled{2}}$$

$$\textcircled{2} \quad - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)$$

$$= - \sum_{i=1}^n \alpha_i y_i (w^T x_i + b) + \sum_{i=1}^n \alpha_i$$

$$= - \sum_{i=1}^n \alpha_i y_i w^T x_i - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i$$

$$= - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^n \alpha_i$$

RECALL

$$\frac{\partial \mathcal{L}(w, b, \alpha)}{\partial b} = 0 \quad \Longrightarrow \quad \sum_{i=1}^n \alpha_i y_i = 0$$

$$\frac{\partial \mathcal{L}(w, b, \alpha)}{\partial w} = 0 \quad \Longrightarrow \quad w = \sum_{i=1}^n \alpha_i y_i x_i$$

(참고 : https://youtu.be/qFg8cDnqYCI?si=w51RhK05oRE_Gd-3)

03 | Support Vector Machine

Primal to Dual

- Dual problem

- ✓ 따라서 Lagrangian dual은 다음과 같은 QP문제로 바뀌게 됨

- ✓
$$\underset{\alpha}{\text{maximize}} \sum_{i=1}^n \alpha_i - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$\begin{aligned} \text{subject to } & \sum_{i=1}^n \alpha_i y_i = 0, \\ & \alpha_i \geq 0, i = 1, 2, \dots, n \end{aligned}$$

- Why Dual problem?

- ✓ Optimization 문제가 x 들의 내적만으로 표현됨 → Kernel Trick이 가능해짐

- ✓ $w = \sum \alpha_i y_i x_i$, 결정경계가 데이터들의 선형 결합으로 표현 되어 Support Vector만 남는 구조가 보임

03 | Support Vector Machine

KKT Condition

- KKT Condition

- ✓ Stationarity

- $\frac{\partial L(w,b,\alpha)}{\partial w} = 0 \rightarrow \sum_{i=1}^n \alpha_i y_i x_i$, $\frac{\partial L(w,b,\alpha)}{\partial b} = 0 \rightarrow \sum_{i=1}^n \alpha_i y_i = 0$

- ✓ Primal feasibility

- $y_i(w^T x_i + b) \geq 1, i = 1, 2, \dots, n$

- ✓ Dual feasibility

- $\alpha_i \geq 0, i = 1, 2, \dots, n$

- ✓ Complementary slackness

- $\alpha_i (y_i(w^T x_i + b) - 1) = 0$

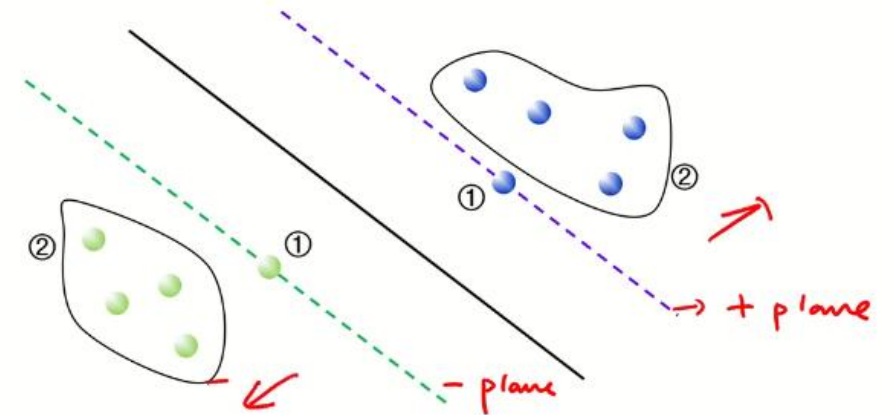
- Complementary slackness

- ① $\alpha_i > 0$ & $y_i(w^T x_i + b) - 1 = 0 \rightarrow x_i$ 가 margin에 위에 있음 (Support Vector)

- ② $\alpha_i = 0$ & $y_i(w^T x_i + b) - 1 \neq 0 \rightarrow x_i$ 가 margin에 위에 있지 않음 (Hyperplane을 구축하는데 영향을 미치지 않음)

- ✓ x_i 가 support vector 인 경우에 $\alpha_i > 0$ 이므로 $w^* = \sum_{i=1}^n \alpha_i^* y_i x_i = \sum_{i \in SV} \alpha_i^* y_i x_i$

- ✓ 즉, support vector만 이용하여 optimal hyperplane을 구할 수 있음



03 | Support Vector Machine

Soft Margin

- 분리 불가능한 선형 SVM

- ✓ 다음의 경우 선형함수로 두 클래스를 분리하는 것이 불가능
- ✓ 분리 불가능한 선형 SVM을 학습하기 위해 해당 클래스의 hyperplane과의 거리를 나타내는 ξ 를 도입

- ✓ Minimize $\frac{\|w\|^2}{2} + C \sum_{i=1}^n \xi_i$
Subject to

마진 최대화

오류 최소화

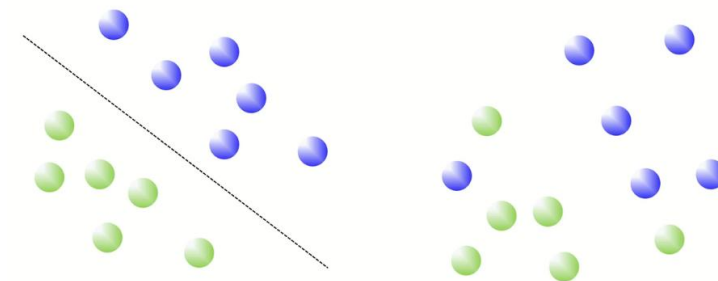
$$\begin{aligned} w^T x_i + b &\geq 1 - \xi_i \quad \text{for } y_i = 1 \\ w^T x_i + b &\leq -1 + \xi_i \quad \text{for } y_i = -1 \end{aligned}$$

$$\rightarrow y_i(w^T x_i + b) \geq 1 - \xi_i \quad \text{for } \forall i = (1, 2, \dots, n)$$

(참고 : 데이터 마이닝 SVM 강의자료)

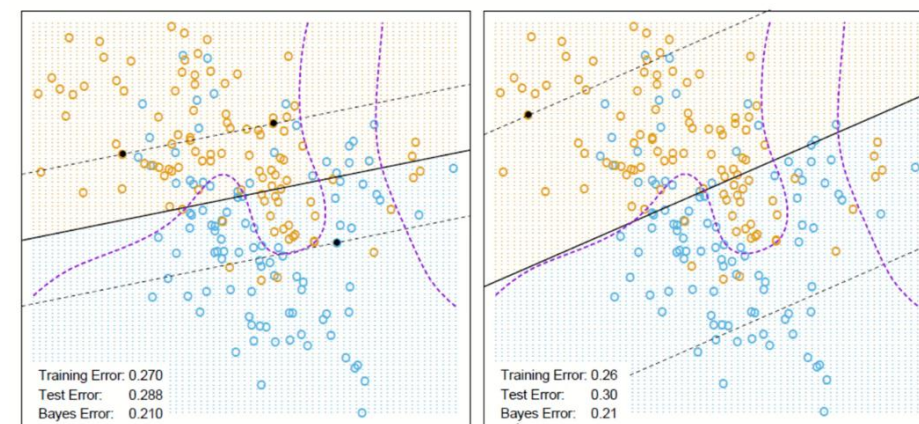
- ✓ C는 Margine과 training error에 대한 trade-off를 결정하는 parameter

- C↑ : training error를 많이 허용하지 않음 → overfit
- C↓ : training error 많이 허용 → underfit



Linearly separable

Linearly nonseparable



C = 10000

C = 0.01

03 | Support Vector Machine

Soft Margin

- Soft Margin

- ✓ Soft margin을 위한 해당 라그랑주는 다음과 같음

- ✓ $\mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}, \boldsymbol{\xi}, \boldsymbol{\mu}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{n=1}^N \xi_n - \sum_{n=1}^N \alpha_n (\tilde{y}_n (\mathbf{w}^T \mathbf{x}_n + w_0) - 1 + \xi_n) - \sum_{n=1}^N \mu_n \xi_n$

- ✓ dual 형식으로 표현 $\mathcal{L}(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \mathbf{x}_i^T \mathbf{x}_j$

- KKT Condition

- ✓ $0 \leq \alpha_n \leq C$

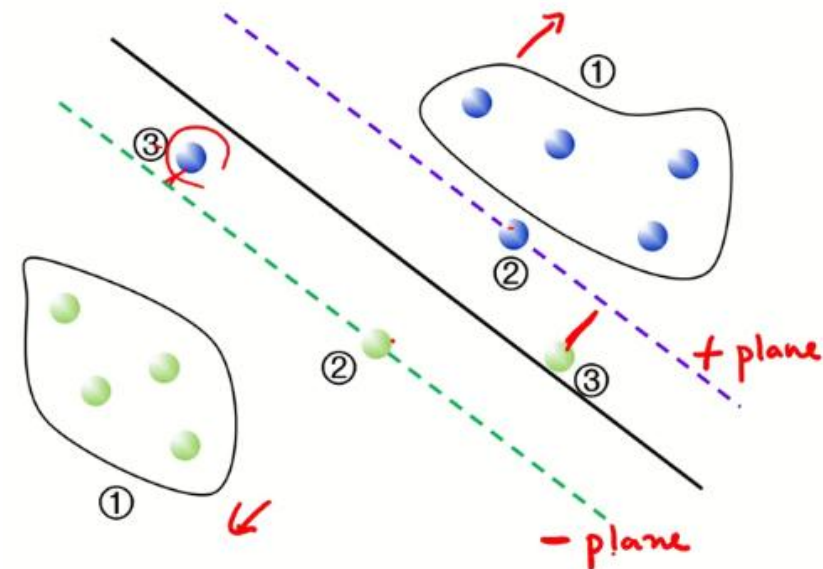
- ✓ $\sum_{n=1}^N \alpha_n \tilde{y}_n = 0$

- Meaning of Alpha

- ① $\alpha_n = 0 \rightarrow$ Margin 밖으로 충분히 멀리 있음

- ② $0 < \alpha_n < C \rightarrow$ Slack이 없으며, 정확히 Margin 위에 있음

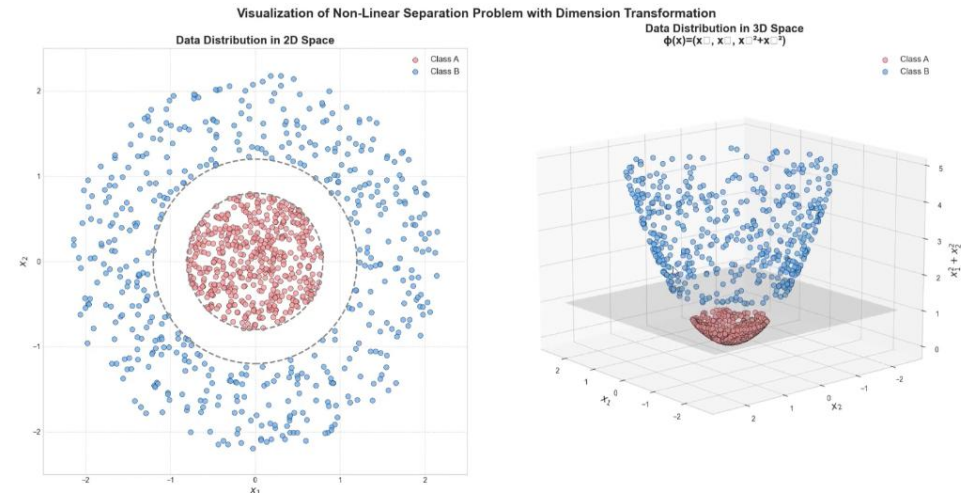
- ③ $\alpha_n = C \rightarrow$ 위반한 점



03 | Support Vector Machine

Kernel Trick

- Mapping Original Space to Kernel Space
 - ✓ SVM을 original space가 아닌 feature space에서 학습
 - ✓ Original space에서 nonlinear decision boundary → Feature space에서 linear decision boundary
 - ✓ 고차원 feature space에서는 관측치 분류가 더 쉬울 수 있음



- Kernel Trick
 - ✓ Dual Problem의 원론적인 이점은 모든 내적 연산 $x^T x'$ 을 양의 정 부호 커널 함수 $K(x, x')$ 에 대한 호출로 바꿀 수 있음
 - ✓ 원래는 데이터를 일일이 고차원으로 변환 후 고차원 공간에서 두 데이터의 내적을 구해야 함
 - ✓ 하지만 원래 차원에서 두 데이터를 커널 함수 $K(x, x')$ 에 넣으면 마치 고차원으로 변환해서 내적을 구한 것과 같음

$$\begin{aligned} & \underset{\alpha}{\text{maximize}} \quad \sum_{i=1}^n \alpha_i - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \\ & \text{subject to} \quad \sum_{i=1}^n \alpha_i y_i = 0, \\ & \quad \alpha_i \geq 0, i = 1, 2, \dots, n \end{aligned}$$

03 | Support Vector Machine

Sparse Vector Machines

- Sparse Vector Machines
 - ✓ SVM은 Support Vector를 이용해서 계산이 빠르지만 확률을 출력하지는 않음
 - ✓ Sparse Vector Machines은 확률 또한 출력하고자 함
- Idea
 - ✓ Feature를 새 데이터 x 에 대해 원래 좌표가 아닌 각 훈련데이터와 유사도의 벡터로 바꿔서 표현
 - ✓ $\phi(x) = [K(x, x_1), \dots, K(x, x_N)]$
 - ✓ 유사도의 벡터로 바꾼 후 로지스틱 회귀 등을 얹어 확률적으로 표현
 - ✓ $p(y_n = 1|x_n) = \sigma(w^T \phi(x_n) + b)$
 - ✓ 훈련 데이터가 N 개면 feature도 N 개가 생김 $\rightarrow$ 훈련 데이터가 많아질수록 feature도 많아져서 과적합 유발
 - ✓ l_2 정규화를 사용해 적합 시키면 L2VM이라 부름 $\rightarrow$ 예측 성능 자체는 좋은 경우가 많지만, w 가 sparse하지 않음
 - ✓ w 가 sparse하도록 사전분포를 가하여 모든 exemplars를 유지할 필요가 없도록 만들
 - ✓ 이것을 sparse vector machine 이라 부름

03 | Support Vector Machine

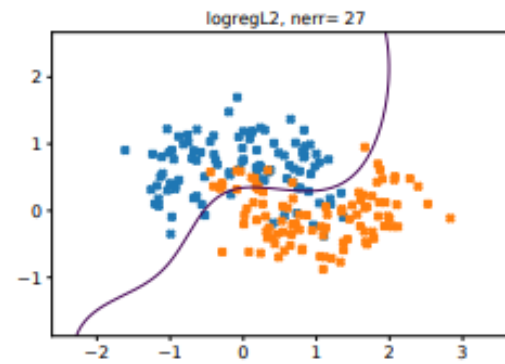
Relevance Vector Machines

- Background
 - ✓ 가중치 w 를 sparse하게 만드는 가장 쉬운 방법은 l_1 정규화를 쓰는 것
 - ✓ 하지만 l_1 정규화만으로는 원하는 정확도를 유지하면서 충분히 sparse하게 만들 수 없음
- ARD(Automatic Relevancy Determination)
 - ✓ Margine0이 아닌 베이지안 방식으로 sparse한 weight vector를 추정
 - ✓ 이는 weight마다 사전확률을 기반으로 중요도를 따로 보게 됨
 - $p(w_i|\alpha_i) = N(0, \alpha_i^{-1})$
 - ✓ ARD 기법을 커널로 만든 feature vector $\phi(x) = [K(x, x_1), \dots, K(x, x_N)]$ 에 적용하면 그 방법을 **RVM**이라 부름

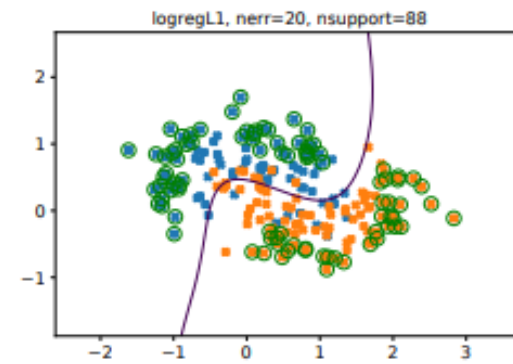
03 | Support Vector Machine

SVM, L1VM, L2VM, RVM

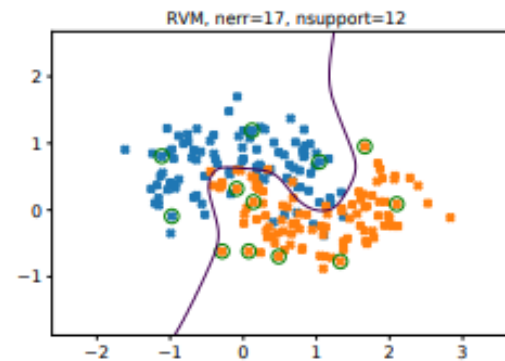
- RBF Kernel
 - ✓ 두 점이 가까우면 큰 값, 멀면 거의 0을 주는 유사도 함수
 - ✓ $K(x, x') = e^{-\frac{|x-x'|}{2\sigma^2}}$
- Comparison
 - ✓ 같은 RBF커널을 쓰더라도 어떤 방법은 모든 데이터에 의존하고 어떤 방법은 일부만 사용함
 - ✓ 즉, 커널이 중요한 게 아닌, 어떤 방식으로 weight를 sparse 하게 만드는지가 알고리즘의 효율을 결정함



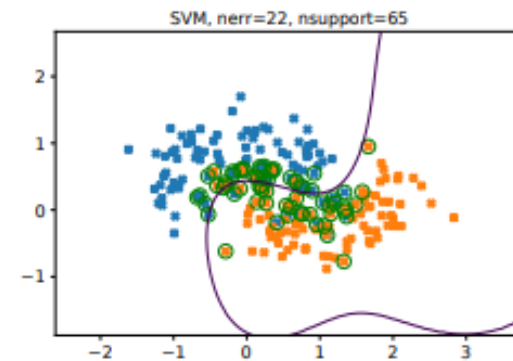
(a)



(b)



(c)



(d)

Q&A

감사합니다.